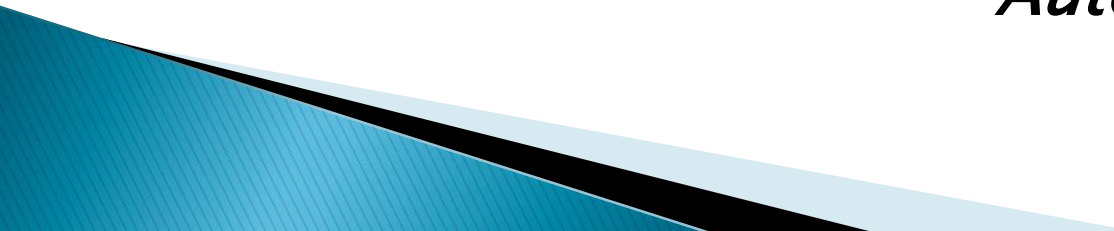


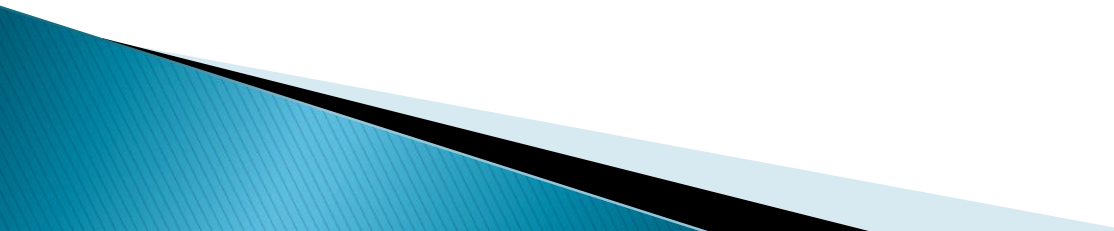
Operatory ANY, ALL, IN, EXIST w podzapytaniach.

Autor: Joanna Karwowska



Operatory ANY, ALL, IN, EXIST

- ▶ Rezultat zwracany przez podzapytanie (zapytanie wewnętrzne) można porównywać za pomocą jednego ze specjalnych operatorów dostępnych w SQL.
- ▶ Możemy za ich pomocą sprawdzić między innymi czy podzapytanie zwróciło jakieś dane lub wartość wiersza.

- ▶ ALL – sprawdza wartości wszystkich wierszy zwróconych przez podzapytanie.
 - ▶ ANY – sprawdza wartość dowolnego wiersza zwróconego przez podzapytanie.
 - ▶ IN – sprawdza, czy wartość należy do pewnego zbioru; równoważnik konstrukcji =ANY.
 - ▶ EXISTS – zwraca true , jeśli podzapytanie zwróciło jakiekolwiek dane, jeśli nie zwraca – false.
- 

Mamy bazę danych zapytania zbudowaną z jednej tabeli:

id	imie	nazwisko	stanowisko	placa
1	Adam	Kowalski	magazynier	2000.00
2	Adam	Nowak	kierownik	8000.00
3	Arkadiusz	Nowicki	kierowca	6000.00
4	Jan	Miodek	pracownik	3000.00
5	Jan	Pajac	pracownik	4000.00
6	Jan	Paj	pracownik	3000.00
7	Arkadiusz	Nowicki	pracownik	5000.00

Przykład 1

Znajdź pracownika otrzymującego najniższą płacę.

```
SELECT imie, nazwisko  
FROM pracownicy  
WHERE placa =  
    (SELECT MIN(placa) FROM pracownicy);
```

Rezultat:

imie	nazwisko
Adam	Kowalski

Operator ANY – warunek jest prawdziwy, jeśli jest spełniony dla przynajmniej jednej wartości zwracanej przez podzapytanie.

Przykład 2

Podaj nazwiska pracowników, których płaca jest większa od płacy dowolnego pracownika na stanowisku pracownik.

```
SELECT imie, nazwisko, placa  
FROM pracownicy WHERE placa > ANY  
(SELECT placa FROM pracownicy  
WHERE stanowisko = 'pracownik');
```

Rezultat:

imie	nazwisko	placa
Adam	Nowak	8000.00
Arkadiusz	Nowicki	6000.00
Jan	Pajac	4000.00
Arkadiusz	Nowicki	5000.00

Operator ALL – warunek jest prawdziwy, jeśli jest spełniony dla wszystkich wartości zwracanych przez podzapytanie.

Przykład 3

Podaj imiona i nazwiska pracowników, których płaca jest wyższa od płacy wszystkich pracowników na stanowisku pracownik.

```
SELECT imie, nazwisko, placa  
FROM pracownicy WHERE placa > ALL  
(SELECT placa FROM pracownicy  
WHERE stanowisko = 'pracownik');
```

Rezultat:

imie	nazwisko	placa
Adam	Nowak	8000.00
Arkadiusz	Nowicki	6000.00

Operator IN (zawierania w zbiorze).

Przykład 4

Podaj pracowników otrzymujących najwyższe płace wśród swojej grupy imion.

```
SELECT imie, nazwisko, placa  
FROM pracownicy WHERE (imie, placa) IN  
(SELECT imie, MAX(placa) FROM pracownicy  
GROUP BY imie);
```

Rezultat:

imie	nazwisko	placa
Adam	Nowak	8000.00
Arkadiusz	Nowicki	6000.00
Jan	Pajac	4000.00

Przykład 5

Podaj pracowników otrzymujących najwyższe płace wśród swojej grupy imion (IN =ANY)

```
SELECT imie, nazwisko, placa  
FROM pracownicy WHERE (imie, placa) = ANY  
(SELECT imie, MAX(placa)  
FROM pracownicy  
GROUP BY imie);
```

Rezultat:

imie	nazwisko	placa
Adam	Nowak	8000.00
Arkadiusz	Nowicki	6000.00
Jan	Pajac	4000.00

Tworzenie zapytań z IN i NOT IN

- ▶ Wyrażenie IN jest wykorzystywane do sprawdzenia, czy wartość należy do pewnego zbioru.
- ▶ Podzapytanie może być wykorzystane do wybrania tego zbioru wartości.

Przykład 6 (hurtownia) – wyszukaj imiona i nazwiska osób, które złożyły zamówienie.

```
SELECT imie, nazwisko  
FROM klienci  
WHERE id IN  
  (SELECT klient_id FROM zamowienia  
   WHERE zamowienia.klient_id = klienci.id );
```

Rezultat:

Imie	Nazwisko
Jan	Kowalski
Andrzej	Nowak
Janusz	Malinowski
Adam	Kowalski
Krzysztof	Nowicki

Przykład 7 – wyszukaj imiona i nazwiska osób, które nie złożyły zamówienia.

```
SELECT imie, nazwisko  
FROM klienci  
WHERE id NOT IN  
  (SELECT klient_id  
   FROM zamowienia  
   WHERE klienci.id = zamowienia.klient_id);
```

Rezultat:

Imie	Nazwisko
Anna	Nowak

Wykorzystanie EXISTS

- ▶ Słowo kluczowe **EXISTS** zostało zaprojektowane specjalnie do wykorzystania w podzapytaniach.
- ▶ Składnia instrukcji wykorzystującej słowo **EXISTS** jest następująca:

SELECT lista **FROM** nazwa_tabeli
WHERE EXISTS (podzapytanie);

- ▶ W przypadku, gdy podzapytanie zwraca dowolną wartość, to klauzula **EXISTS** zwraca wartość logiczną **prawda**.

Wykorzystanie EXISTS – cd.

- ▶ Klauzula **EXISTS** może być wykorzystana również w zapytaniu niepowiązanym.
- ▶ W takim przypadku, gdy podzapytanie zwraca jakiegokolwiek wiersze, klauzula daje wynik prawda, w przeciwnym przypadku fałsz.
- ▶ Klauzula **EXISTS** jest bardzo przydatna w połączeniu z zapytaniami skorelowanymi.
- ▶ Wykonywane są one dla każdego wiersza tabeli, a wartości aktualnie wybranego wiersza są przekazywane do klauzuli **WHERE** podzapytania.
- ▶ Wykorzystując klauzule **WHERE** można porównywać dane z każdego wiersza tabeli z danymi z innych tabel.

Przykład 8

Wyszukaj dane osób, które złożyły zamówienia.

SELECT imie, nazwisko

FROM klienci

WHERE EXISTS

(SELECT klient_id

FROM zamowienia

WHERE klienci.id = zamowienia.klient_id);

Rezultat:

Imie	Nazwisko
Jan	Kowalski
Andrzej	Nowak
Janusz	Malinowski
Adam	Kowalski
Krzysztof	Nowicki

Przykład 9

Wyszukaj dane osób, które nie złożyły zamówienia.

```
SELECT imie, nazwisko
```

```
FROM klienci
```

```
WHERE NOT EXISTS
```

```
  (SELECT klient_id FROM zamowienia
```

```
   WHERE zamowienia.klient_id =klienci.id );
```

- ▶ Klauzula NOT EXISTS jest wykorzystywana do znajdowania wierszy, dla których powiązane podzapytanie nie zwraca żadnych wartości.
- ▶ Przydaje się to do znajdowania wierszy, które nie zawierają powiązanych danych w innych tabelach.

Rezultat:

Imie	Nazwisko
Anna	Nowak

Operator ANY

Przykład 10

Znajdź informacje o towarach (innych niż Płaskowniki), które zamówili klienci.

```
SELECT nazwa FROM towary  
WHERE 'Płaskowniki' <> ANY  
  (SELECT nazwa FROM zamowienia  
   WHERE zamowienia.towar_id = towary.id);
```

Rezultat:

nazwa
Śrubki
Nakrętki

Mamy bazę danych zbudowaną z dwóch tabel:

Tabela: `pracownicy`

PracownicyID (♣)	Miasto (♣)	Imie	Nazwisko	Data_urodzenia	Stanowisko	Data_zatrudnienia
1	1	Jan	Kowalski	1984-12-23	Programista	1999-12-09
2	1	Adam	Adamiak	1991-08-08	Menadżer	2011-31-01
3	3	Jakub	Wawrzyniak	1990-16-03	Programista	2010-01-04
4	2	Michał	Jakubiak	1981-05-02	Projektant	1998-09-22

Gdzie kluczem (♣) jest kolumna `PracownicyID`.

Tabela: `miastoZamieszkania`

MiastoZamieszkaniaID (♣)	Miasto
1	Kraków
2	Warszawa
3	Wrocław
4	Londyn

Gdzie kluczem (♣) jest kolumna `MiastoZamieszkaniaID` oraz (♣).

Obie tabele zostały połączone za pomocą relacji `PracownicyID -> MiastoZamieszkaniaID (♣ -> ♣)`. Oznacza to, że do każdego pracownika zostało przypisane jego miasto zamieszkania.

Ćw. 1 –Pobierzmy z bazy danych informacje o pracownikach, którzy nie mieszkają w Krakowie:

```
SELECT imie, nazwisko FROM pracownicy  
WHERE EXISTS  
(SELECT * FROM miastoZamieszkania  
WHERE  
pracownicy.miasto = miastoZamieszkaniaID  
AND miasto <> 'Kraków');
```

Rezultat:

Imie	Nazwisko
Jakub	Wawrzyniak
Michał	Jakubiak

Ćw. 2 –Pobierzmy z bazy danych informacje o miastach,
w których nie mieszkają żadni pracownicy.

```
SELECT Miasto  
FROM miastoZamieszkania AS mz  
WHERE NOT EXISTS  
(SELECT * FROM pracownicy  
WHERE mz.MiastoZamieszkaniaID = miasto);
```

Rezultat:

Miasto
Londyn

W tym ćwiczeniu użyliśmy przeczenia (NOT), NOT EXISTS jest prawdziwe, jeśli nie zostanie zwrócony przez podzapytanie żaden rekord.

Ćw. 3 –Pobierzmy z bazy danych informacje o pracownikach mieszkających w innych miejscowościach niż Madryt

```
SELECT imie, nazwisko FROM pracownicy  
WHERE 'Madryt' <> ANY  
  (SELECT miasto FROM miastoZamieszkania);
```

Rezultat:

Imię	Nazwisko
Jan	Kowalski
Adam	Adamiak
Jakub	Wawrzyniak
Michał	Jakubiak

Operator ANY zwraca wartość PRAWDA jeśli którekolwiek ze zwróconych przez podzapytanie danych spełni poprzedzający go warunek logiczny.

Ćw. 4 –Pobierzmy z bazy danych informacje o pracownikach mieszkających w innych miejscowościach niż LONDYN:

```
SELECT imie, nazwisko FROM pracownicy  
WHERE 'Londyn' <> ALL  
  (SELECT miasto FROM miastoZamieszkania);
```

Rezultat: 0 rekordów.

Operator ALL działa w zasadzie tak samo jak ANY z tym wyjątkiem, że sprawdza wszystkie dane. Wszystkie zwrócone przez podzapytanie rekordy muszą spełniać podany wcześniej warunek logiczny.

W tabeli miastoZamieszkania znajduje się miasto 'Londyn', dlatego warunek logiczny nie został spełniony.

